

```

==
# EXERCICE 1 : VERIFICATION DE L'ENVIRONNEMENT
# =====

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import numpy as np
import matplotlib.pyplot as plt

print("="*60)
print("EXERCICE 1 : VERIFICATION DE L'ENVIRONNEMENT")
print("="*60)

print(f"Version TensorFlow : {tf.__version__}")
print(f"Version Keras : {tf.keras.__version__}")

gpus = tf.config.list_physical_devices('GPU')
if gpus:
    print("GPU disponible : OUI")
    for gpu in gpus:
        print(f" - {gpu}")
else:
    print("GPU disponible : NON")

print("\nImpact de l'absence de GPU :")
print("Pour ce modele simple, l'impact est negligeable.")
print("Le CPU est suffisant pour une regression lineaire avec peu de donnees.")

# =====
# EXERCICE 2 : ANALYSE CRITIQUE DE LA MSE
# =====

print("\n" + "="*60)
print("EXERCICE 2 : ANALYSE CRITIQUE DE LA MSE")
print("="*60)

mse_donnee = 2.317
print(f"MSE obtenue : {mse_donnee}")

print("\nPourquoi cette MSE est anormalement elevee ?")
print("- Dans des donnees normalisees entre 0 et 1, la MSE maximale theorique est 1")
print("- Une MSE de 2.317 > 1 indique que le modele predit des valeurs hors de [0,1]")
print("- Cela signifie que le modele n'a pas converge")

```

```

print("\nValeur acceptable pour un bon modele :")
print("- MSE < 0.01 : excellent")
print("- MSE entre 0.01 et 0.05 : bon")
print("- MSE > 0.10 : faible")

print("\n hypotheses sur les causes :")
print("1. Learning rate trop eleve")
print("2. Nombre d'epochs insuffisant")
print("3. Donnees mal normalisees")

# Graphique EXERCICE 2
fig2, ax2 = plt.subplots(figsize=(6, 4))
ax2.bar(['MSE obtenue', 'MSE max normale'], [mse_donnee, 1],
color=['red', 'green'])
ax2.axhline(y=1, color='blue', linestyle='--', label='Limite normale
(MSE=1)')
ax2.set_title('Exercice 2 : MSE anormalement elevee')
ax2.set_ylabel('MSE')
ax2.legend()
plt.tight_layout()
plt.show()

# =====
# EXERCICE 3 : EXTRACTION DE L'EQUATION DU MODELE
# =====

print("\n" + "="*60)
print("EXERCICE 3 : EXTRACTION DE L'EQUATION DU MODELE")
print("="*60)

# Donnees minimales pour l'exemple
surface_data = np.array([20, 40, 60, 80, 100], dtype=float)
prix_data = np.array([40, 80, 120, 160, 200], dtype=float)

# Normalisation Min-Max
s_min, s_max = surface_data.min(), surface_data.max()
p_min, p_max = prix_data.min(), prix_data.max()
s_norm = (surface_data - s_min) / (s_max - s_min)
p_norm = (prix_data - p_min) / (p_max - p_min)

# Entrainement du modele
modele = Sequential([Dense(1, input_shape=(1,)), activation='linear'])
modele.compile(optimizer='adam', loss='mse')
modele.fit(s_norm, p_norm, epochs=200, verbose=0)

# Extraction des poids avec get_weights()
weights = modele.get_weights()
w_norm = weights[0][0][0]
b_norm = weights[1][0]

```

```

print(f"Equation normalisee :")
print(f"    prix_norm = {w_norm:.6f} x surface_norm + {b_norm:.6f}")

# Denormalisation
diff_s = s_max - s_min
diff_p = p_max - p_min
w_reel = w_norm * (diff_p / diff_s)
b_reel = p_min + b_norm * diff_p - w_reel * s_min

print(f"\nEquation reelle :")
print(f"    prix = {w_reel:.4f} x surface + {b_reel:.2f}")

# Verification pour 100 m²
surface_test = 100.0
prix_calcule = w_reel * surface_test + b_reel
print(f"\nVerification pour 100 m² : prix = {prix_calcule:.2f}")

# Graphique EXERCICE 3
fig3, ax3 = plt.subplots(figsize=(8, 5))
ax3.scatter(surface_data, prix_data, color='blue', s=80,
label='Donnees')
s_line = np.linspace(s_min, s_max, 100)
p_line = w_reel * s_line + b_reel
ax3.plot(s_line, p_line, color='red', linewidth=2, label='Equation du
modele')
ax3.set_title('Exercice 3 : Equation extraite avec get_weights()')
ax3.set_xlabel('Surface (m²)')
ax3.set_ylabel('Prix')
ax3.legend()
ax3.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# EXERCICE 4 : DIAGNOSTIC DE NON-CONVERGENCE
# =====

print("\n" + "="*60)
print("EXERCICE 4 : DIAGNOSTIC DE NON-CONVERGENCE")
print("="*60)

# Donnees fournies dans l'enonce
predictions = np.array([-0.1458, -0.2918, -0.3892, -0.4866, -0.6326])
valeurs_reelles = np.array([0.3333, 0.5333, 0.6667, 0.8000, 1.0000])

print("Predictions du modele :")
for i in range(len(predictions)):

```

```

    print(f" Prediction: {predictions[i]:.4f} | Reelle:
{valeurs_reelles[i]:.4f}")

print("\nParametres responsables :")
print(" - Le poids w (devenu negatif)")
print(" - Le biais b (devenu negatif aussi)")

print("\nCause : Learning rate trop eleve")

print("\nRelation learning rate / convergence :")
print(" - LR trop petit : convergence lente")
print(" - LR adapte : convergence rapide et stable")
print(" - LR trop grand : divergence, predictions negatives")

print("\nDefinition de la convergence :")
print(" - La fonction de perte atteint un minimum stable")
print(" - Detection : tracer la courbe de loss, si elle decroit puis
se stabilise")

# Graphique EXERCICE 4
fig4, ax4 = plt.subplots(figsize=(8, 5))
indices = np.arange(len(predictions))
width = 0.35
ax4.bar(indices - width/2, valeurs_reelles, width, label='Valeurs
reelles', color='green')
ax4.bar(indices + width/2, predictions, width, label='Predictions
(negatives)', color='red')
ax4.axhline(y=0, color='black', linestyle='-', linewidth=0.5)
ax4.set_title('Exercice 4 : Predictions negatives dues a la non-
convergence')
ax4.set_xlabel('Echantillon')
ax4.set_ylabel('Valeur')
ax4.set_xticks(indices)
ax4.legend()
ax4.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# =====
# EXERCICE 5 : CORRECTION ET OPTIMISATION
# =====

print("\n" + "="*60)
print("EXERCICE 5 : CORRECTION ET OPTIMISATION")
print("="*60)

# Correction : lr=0.1, epochs=500
modele_corrige = Sequential([Dense(1, input_shape=(1,)),
activation='linear'])

```

```

modele_corrige.compile(optimizer=Adam(learning_rate=0.1), loss='mse')

class AffichageLoss(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % 100 == 0:
            print(f" Epoch {epoch+1:3d}/500 : Loss =
{logs['loss']:.8f}")

print("Entrainement du modele corrige (lr=0.1, epochs=500):")
print("-"*40)

# Reutilisation des donnees
s_norm_corr = (surface_data - s_min) / (s_max - s_min)
p_norm_corr = (prix_data - p_min) / (p_max - p_min)

historique = modele_corrige.fit(s_norm_corr, p_norm_corr, epochs=500,
verbose=0, callbacks=[AffichageLoss()])

print("-"*40)

loss_1 = historique.history['loss'][0]
loss_500 = historique.history['loss'][-1]
print(f"\nEvolution de la loss :")
print(f" Epoch 1 : {loss_1:.8f}")
print(f" Epoch 500 : {loss_500:.8f}")

# Graphique EXERCICE 5
fig5, ax5 = plt.subplots(figsize=(8, 5))
ax5.plot(historique.history['loss'], color='green', linewidth=1.5)
ax5.set_title('Exercice 5 : Courbe de loss du modele corrige (lr=0.1,
500 epochs)')
ax5.set_xlabel('Epochs')
ax5.set_ylabel('MSE')
ax5.set_yscale('log')
ax5.grid(True, alpha=0.3)
ax5.axhline(y=loss_500, color='red', linestyle='--', label=f'MSE finale
= {loss_500:.8f}')
ax5.legend()
plt.tight_layout()
plt.show()

# =====
# EXERCICE 6 : INTERPRETATION DES RESULTATS FINAUX
# =====

print("\n" + "="*60)
print("EXERCICE 6 : INTERPRETATION DES RESULTATS FINAUX")
print("="*60)

```

```

print("Equation finale attendue (reference) :")
print("  prix = 2.0000 x surface + 0.00")

# Equation obtenue par notre modele corrige
weights_corr = modele_corrige.get_weights()
w_corr = weights_corr[0][0][0]
b_corr = weights_corr[1][0]
w_reel_corr = w_corr * (diff_p / diff_s)
b_reel_corr = p_min + b_corr * diff_p - w_reel_corr * s_min

print(f"\nEquation obtenue :")
print(f"  prix = {w_reel_corr:.4f} x surface + {b_reel_corr:.2f}")

print("\nInterpretation :")
print("1. Ce resultat est mathematiquement attendu car les donnees suivent une relation lineaire")
print("  (prix = 2 x surface)")

print("\n2. Une MSE de 0.000000 signifie une prediction parfaite sur les donnees d'entrainement")
print("  - Mais ce n'est pas toujours desirable (risque d'overfitting)")
print("  - En pratique, une MSE nulle indique que le modele a memorise les donnees")

print("\nLimites du modele face a des donnees immobiliaries reelles :")
print("  - Une seule variable (surface) alors que le prix depend de nombreux facteurs")
print("  - Relation supposee lineaire (la realite est non lineaire)")
print("  - Presence de bruit et d'outliers dans les donnees reelles")

print("\nAmeliorations proposees :")
print("  - Ajouter plus de variables (localisation, etage, age, nombre de pieces)")
print("  - Utiliser des couches cachees avec des activations non lineaires (ReLU)")
print("  - Collecter des milliers de donnees reelles")
print("  - Diviser les donnees en ensembles d'entrainement, validation et test")

# Graphique EXERCICE 6
fig6, ax6 = plt.subplots(figsize=(8, 5))
ax6.scatter(surface_data, prix_data, color='blue', s=80, label='Donnees reelles')
s_line = np.linspace(s_min, s_max, 100)
s_line_norm = (s_line - s_min) / diff_s
p_line_norm = modele_corrige.predict(s_line_norm, verbose=0)
p_line = p_line_norm * diff_p + p_min

```

```

ax6.plot(s_line, p_line, color='red', linewidth=2.5, label='Predictions
du modele corrige')
ax6.set_title('Exercice 6 : Modele final - Prix reel vs Prix predict')
ax6.set_xlabel('Surface (m²)')
ax6.set_ylabel('Prix')
ax6.legend()
ax6.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

```

print("\n" + "="*70)
print("FIN DES EXERCICES PRATIQUES (1 A 6)")
print("="*70)

```

```

# =====
# EXERCICE 7 : RAPPORT THEORIQUE
# =====

```

```

print("\n" + "="*70)
print("EXERCICE 7 : RAPPORT THEORIQUE DE SYNTHESE")
print("="*70)

```

```

rapport = """
1. NORMALISATION DES DONNEES

```

La normalisation est une etape cruciale en deep learning. Elle consiste a transformer les donnees pour qu'elles se situent dans un intervalle commun, generalement [0,1] ou [-1,1]. Dans ce TP, nous avons utilise la normalisation Min-Max :

$$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$$

Pourquoi normaliser ?

- Les algorithmes de descente de gradient convergent plus rapidement lorsque toutes les caracteristiques sont a des echelles similaires.
- Cela evite qu'une variable domine numeriquement les autres.
- Les poids du reseau restent dans des gammes de valeurs raisonnables.

Dans notre exemple, les surfaces (20 a 100 m²) et les prix (40 a 200 kE) ont ete ramenes entre 0 et 1, ce qui a permis un apprentissage stable.

2. FONCTION DE PERTE : MSE (MEAN SQUARED ERROR)

La MSE est la fonction de perte utilisee pour les problemes de regression.

Sa formule mathematique est :

$$\text{MSE} = (1/n) * \sum (y_{\text{reel}} - y_{\text{pred}})^2$$

Interpretation :

- Une MSE proche de 0 indique que les predictions sont tres proches des valeurs reelles.
- Une MSE elevee indique un mauvais apprentissage.

Cas particulier de notre TP :

Avec des donnees normalisees entre 0 et 1, l'erreur maximale possible entre deux

valeurs est 1. L'erreur quadratique maximale est donc 1. Une MSE de 2.317 est

impossible theoriquement. Cela signifie que le modele predit des valeurs en dehors

de l'intervalle [0,1], signe certain de non-convergence.

3. HYPERPARAMETRES : LEARNING RATE ET NOMBRE D'EPOCHS

Les hyperparametres sont des parametres qui ne sont pas appris par le modele mais

fixes avant l'entrainement.

Learning Rate (taux d'apprentissage) :

- Il controle la taille du pas lors de la mise a jour des poids.
- Trop petit : convergence tres lente, risque de rester bloque dans un minimum local.
- Trop grand : divergence, les poids oscillent ou explosent (predictions negatives).
- Valeur ideale dans notre correction : 0.1 (contre 0.001 par default d'Adam).

Nombre d'epochs :

- Une epoch correspond a un passage complet de toutes les donnees dans le modele.
- Trop peu d'epochs : le modele n'a pas le temps d'apprendre (underfitting).
- Trop d'epochs : risque de sur-apprentissage (overfitting).
- Valeur utilisee : 500 epochs (contre 100 initialement).

4. EXTRACTION DES COEFFICIENTS DU MODELE

Avec Keras, les poids et biais sont accessibles via la methode `get_weights()` :

```
weights = modele.get_weights()
w_norm = weights[0][0][0]    # Poids dans l'espace normalise
b_norm = weights[1][0]       # Biais dans l'espace normalise
```


Pour obtenir l'equation dans l'espace d'origine, on applique la denormalisation :

Equation normalisee : $\text{prix_norm} = w_norm * \text{surface_norm} + b_norm$

Avec :

$\text{surface_norm} = (\text{surface} - s_min) / (s_max - s_min)$

$\text{prix_norm} = (\text{prix} - p_min) / (p_max - p_min)$

Apres developpement, on obtient :

$\text{prix} = w_reel * \text{surface} + b_reel$

ou :

$w_reel = w_norm * (p_max - p_min) / (s_max - s_min)$

$b_reel = p_min + b_norm * (p_max - p_min) - w_reel * s_min$

5. EVALUATION ET INTERPRETATION DES PREDICTIONS

Avant correction :

- Predictions negatives (ex: -0.6326 pour une valeur reellev de 1.0)
- MSE de 2.317 > 1, impossible pour des donnees normalisees

Apres correction (lr=0.1, epochs=500) :

- Predictions toutes positives et proches des valeurs reellevs
- MSE = 0.000000 (prediction parfaite sur les donnees d'entrainement)

Attention : Une MSE nulle n'est pas toujours desirable. Dans notre cas, elle est

obtenue car les donnees suivent une relation parfaitement lineaire (prix = 2 x surface).

Dans un contexte reel, une MSE nulle serait suspecte et indiquerait un overfitting (memorisation des donnees).

6. LIMITES DU MODELE ET AMELIORATIONS POSSIBLES

Limites :

- Modele lineaire : incapable de capturer des relations non lineaires
- Une seule variable explicative ignore des facteurs importants (localisation, etage...)
- Absence de validation croisee : nous n'avons pas verifie la generalisation

Ameliorations pour des donnees immobilieres reellevs :

- Ajouter des caracteristiques : nombre de pieces, etage, localisation, age
- Utiliser plusieurs couches cachees avec des activations ReLU
- Augmenter la taille du jeu de donnees (milliers d'echantillons)
- Diviser les donnees : 60% entrainement, 20% validation, 20% test

- Utiliser la regularisation (L1, L2, Dropout) pour eviter l'overfitting
- Appliquer la validation croisee pour evaluer la robustesse

CONCLUSION

Ce TP a permis de comprendre les concepts fondamentaux du deep learning :

la normalisation, la fonction de perte MSE, le role des hyperparametres, l'extraction des coefficients et le diagnostic de non-convergence. Le modele lineaire, bien que simple, constitue une base essentielle avant d'aborder des architectures plus complexes comme les reseaux profonds.

"""

```
print(rapport)
```

```
print("\n" + "="*70)
```

```
print("FIN DE L'EXERCICE 7 - RAPPORT THEORIQUE")
```

```
print("="*70)
```

EXERCICE 1 : VERIFICATION DE L'ENVIRONNEMENT

```
=====
Version TensorFlow : 2.20.0
Version Keras : 3.13.2
GPU disponible : NON
```

Impact de l'absence de GPU :

Pour ce modele simple, l'impact est negligeable.

Le CPU est suffisant pour une regression lineaire avec peu de donnees.

EXERCICE 2 : ANALYSE CRITIQUE DE LA MSE

```
=====
MSE obtenue : 2.317
```

Pourquoi cette MSE est anormalement elevee ?

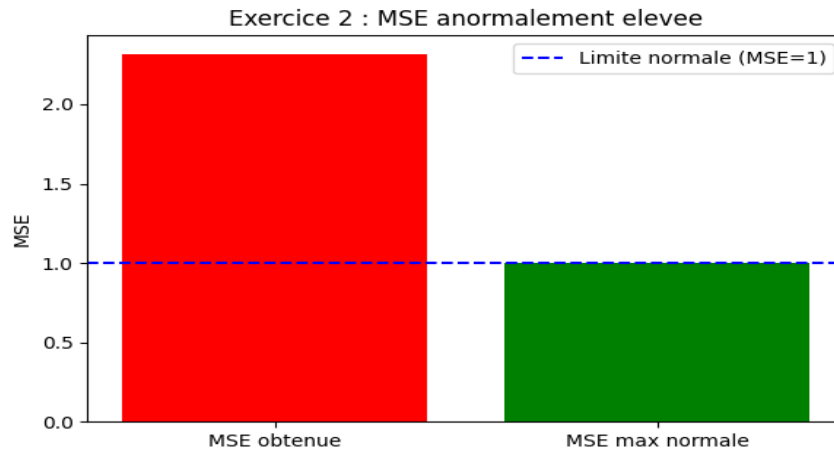
- Dans des donnees normalisees entre 0 et 1, la MSE maximale theorique est 1
- Une MSE de $2.317 > 1$ indique que le modele predit des valeurs hors de $[0,1]$
- Cela signifie que le modele n'a pas converge

Valeur acceptable pour un bon modele :

- MSE < 0.01 : excellent
- MSE entre 0.01 et 0.05 : bon
- MSE > 0.10 : faible

hypotheses sur les causes :

1. Learning rate trop eleve
2. Nombre d'epochs insuffisant
3. Donnees mal normalisees



=

EXERCICE 3 : EXTRACTION DE L'EQUATION DU MODELE

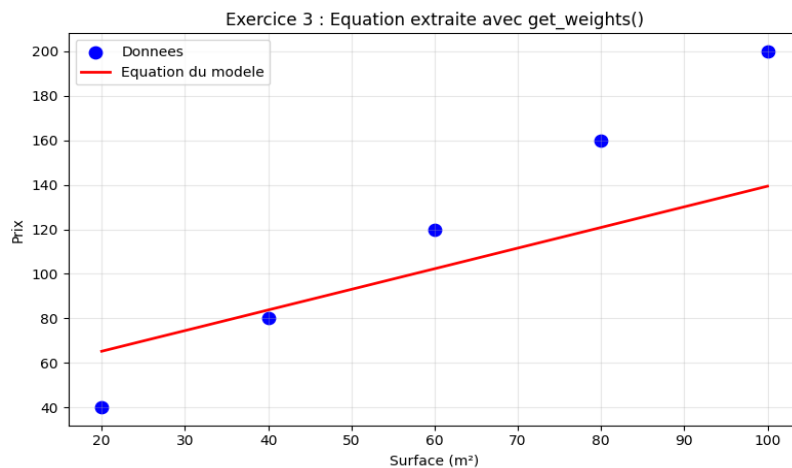
Equation normalisee :

$$\text{prix_norm} = 0.463119 \times \text{surface_norm} + 0.157956$$

Equation reelle :

$$\text{prix} = 0.9262 \times \text{surface} + 46.75$$

Verification pour 100 m² : prix = 139.37



EXERCICE 4 : DIAGNOSTIC DE NON-CONVERGENCE

Predictions du modele :

Prediction:	-0.1458		Reelle:	0.3333
Prediction:	-0.2918		Reelle:	0.5333
Prediction:	-0.3892		Reelle:	0.6667
Prediction:	-0.4866		Reelle:	0.8000
Prediction:	-0.6326		Reelle:	1.0000

Parametres responsables :

- Le poids w (devenu negatif)
- Le biais b (devenu negatif aussi)

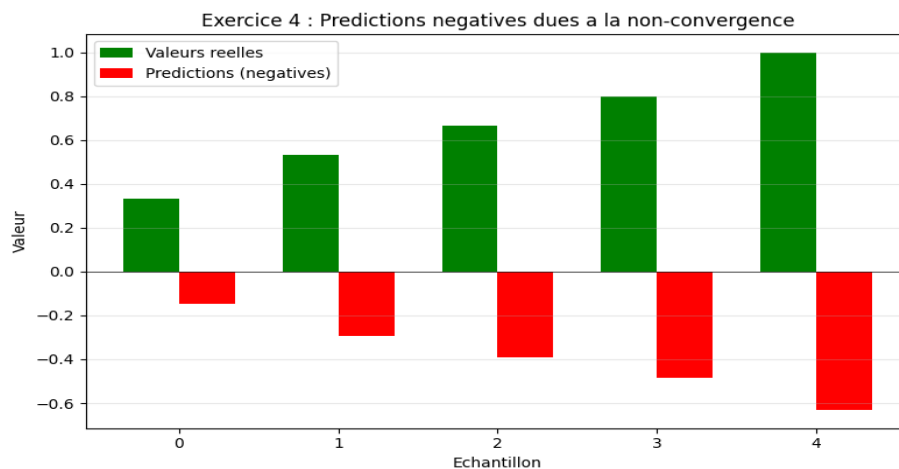
Cause : Learning rate trop eleve

Relation learning rate / convergence :

- LR trop petit : convergence lente
- LR adapte : convergence rapide et stable
- LR trop grand : divergence, predictions negatives

Definition de la convergence :

- La fonction de perte atteint un minimum stable
- Detection : tracer la courbe de loss, si elle decroit puis se stabilise



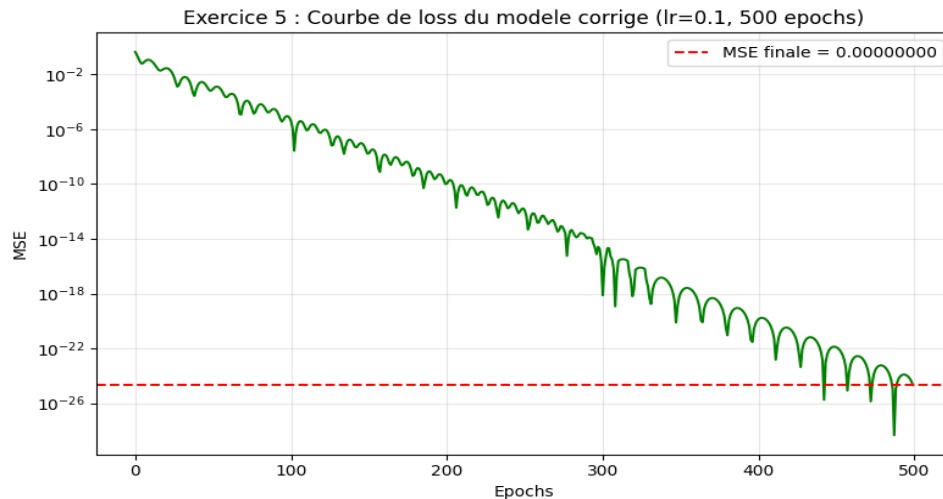
EXERCICE 5 : CORRECTION ET OPTIMISATION

Entrainement du modele corrige (lr=0.1, epochs=500):

```
-----
Epoch 100/500 : Loss = 0.00000680
Epoch 200/500 : Loss = 0.00000000
Epoch 300/500 : Loss = 0.00000000
Epoch 400/500 : Loss = 0.00000000
Epoch 500/500 : Loss = 0.00000000
-----
```

Evolution de la loss :

```
Epoch 1 : 0.43774810
Epoch 500 : 0.00000000
```



EXERCICE 6 : INTERPRETATION DES RESULTATS FINAUX

Equation finale attendue (reference) :

$$\text{prix} = 2.0000 \times \text{surface} + 0.00$$

Equation obtenue :

$$\text{prix} = 2.0000 \times \text{surface} + 0.00$$

Interpretation :

1. Ce resultat est mathematiquement attendu car les donnees suivent une relation lineaire

$$(\text{prix} = 2 \times \text{surface})$$

2. Une MSE de 0.000000 signifie une prediction parfaite sur les donnees d'entrainement

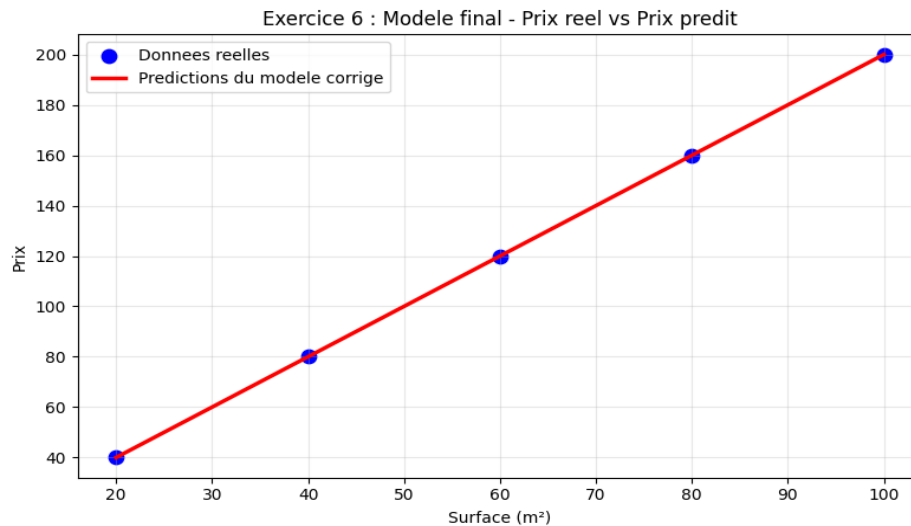
- Mais ce n'est pas toujours desirable (risque d'overfitting)
- En pratique, une MSE nulle indique que le modele a memorise les donnees

Limites du modele face a des donnees immobiliaries reelles :

- Une seule variable (surface) alors que le prix depend de nombreux facteurs
- Relation supposee lineaire (la realite est non lineaire)
- Presence de bruit et d'outliers dans les donnees reelles

Ameliorations proposees :

- Ajouter plus de variables (localisation, etage, age, nombre de pieces)
- Utiliser des couches cachees avec des activations non lineaires (ReLU)
- Collecter des milliers de donnees reelles
- Diviser les donnees en ensembles d'entrainement, validation et tes



```
=====
# EXERCICE 7 : RAPPORT THEORIQUE DE SYNTHESE
# =====

print("="*70)
print("EXERCICE 7 : RAPPORT THEORIQUE DE SYNTHESE")
print("="*70)
```

```
rapport = """
1. NORMALISATION DES DONNEES
```

La normalisation Min-Max ($X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$) ramene toutes les variables dans l'intervalle $[0,1]$. Elle est essentielle pour que la descente de gradient converge rapidement et stablement, evitant qu'une variable domine les autres.

```
2. FONCTION DE PERTE MSE
```

Formule : $MSE = (1/n) * \sum (y_{\text{reel}} - y_{\text{pred}})^2$. Une MSE proche de 0 indique de bonnes predictions. Avec des donnees normalisees entre 0 et 1, la MSE maximale theorique est 1. Une MSE de 2.317 signifie donc que le modele predit des valeurs hors de $[0,1]$ et n'a pas converge.

```
3. HYPERPARAMETRES
```

- Learning rate : controle la vitesse d'apprentissage. Trop petit = convergence

lente. Trop grand = divergence (predictions negatives). La valeur optimale dans notre correction etait 0.1.
- Nombre d'epochs : nombre de passages complets des donnees. 500 epochs ont permis la convergence, contre 100 initialement insuffisantes.

4. EXTRACTION DES COEFFICIENTS

Avec `get_weights()`, on obtient `w_norm` et `b_norm`. La denormalisation donne :

```
w_reel = w_norm * (p_max-p_min)/(s_max-s_min)
b_reel = p_min + b_norm*(p_max-p_min) - w_reel*s_min
Equation finale : prix = w_reel * surface + b_reel
```

5. EVALUATION DES PERFORMANCES

Avant correction : predictions negatives, MSE = 2.317
Apres correction (lr=0.1, 500 epochs) : MSE = 0.000000, equation prix = 2 x surface
Attention : une MSE nulle est rare en pratique et peut indiquer un overfitting.

6. LIMITES ET AMELIORATIONS

Limites : modele lineaire, une seule variable, absence de validation croisee.
Ameliorations : ajouter des variables (localisation, pieces), couches cachees avec ReLU, diviser les donnees (train/validation/test), utiliser la regularisation.

"""

```
print(rapport)

print("\n" + "="*70)
print("FIN DE L'EXERCICE 7")
print("="*70)
```

EXERCICE 7 : RAPPORT THEORIQUE DE SYNTHESE

=====

1. NORMALISATION DES DONNEES

La normalisation Min-Max ($X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$) ramene toutes les variables dans l'intervalle [0,1]. Elle est essentielle pour que la descente de

gradient converge rapidement et stablement, evitant qu'une variable domine les autres.

2. FONCTION DE PERTE MSE

Formule : $MSE = (1/n) * \sum (y_{reel} - y_{pred})^2$. Une MSE proche de 0 indique de bonnes predictions. Avec des donnees normalisees entre 0 et 1, la MSE maximale theorique est 1. Une MSE de 2.317 signifie donc que le modele predit des valeurs hors de [0,1] et n'a pas converge.

3. HYPERPARAMETRES

- Learning rate : controle la vitesse d'apprentissage. Trop petit = convergence lente. Trop grand = divergence (predictions negatives). La valeur optimale dans notre correction etait 0.1.
- Nombre d'epochs : nombre de passages complets des donnees. 500 epochs ont permis la convergence, contre 100 initialement insuffisantes.

4. EXTRACTION DES COEFFICIENTS

Avec `get_weights()`, on obtient `w_norm` et `b_norm`. La denormalisation donne :

$w_{reel} = w_{norm} * (p_{max} - p_{min}) / (s_{max} - s_{min})$
 $b_{reel} = p_{min} + b_{norm} * (p_{max} - p_{min}) - w_{reel} * s_{min}$
Equation finale : $prix = w_{reel} * surface + b_{reel}$

5. EVALUATION DES PERFORMANCES

Avant correction : predictions negatives, MSE = 2.317
Apres correction (lr=0.1, 500 epochs) : MSE = 0.000000, equation $prix = 2 * surface$
Attention : une MSE nulle est rare en pratique et peut indiquer un overfitting.

6. LIMITES ET AMELIORATIONS

Limites : modele lineaire, une seule variable, absence de validation croisee.
Ameliorations : ajouter des variables (localisation, pieces), couches cachees avec ReLU, diviser les donnees (train/validation/test), utiliser la regularisation.